

CENTRO DE BACHILLERATO TECNOLÓGICO INDUSTRIAL Y DE SERVICIOS NO. 182

"José Vasconcelos" - Zinapécuaro, Mich.

PRÁCTICA 2

"GESTIÓN DE PROYECTOS UTILIZANDO REPOSITORIOS"

MÓDULO: M2S1 - Emplea Frameworks para el Desarrollo de Software

Profesor: Jaime Peregrino

Integrantes:
- Alexandra Marisol Jacobo Herrera
- Karen Jacquelin Guillén Ramírez

Grado y Grupo: 3° Semestre - Grupo B

Fecha de Entrega: 12 de Octubre de 2026

ÍNDICE GENERAL

1. Introducción 3

2. Objetivo de la Práctica 3

3. Materiales y Requerimientos del Sistema 3

4. Desarrollo de la Práctica 4

   4.1 Paso 1: Creación de la Cuenta en GitHub 4

   4.2 Paso 2: Instalación y Configuración Inicial de Git 4

   4.3 Paso 3: Creación y Ajuste de Nombre del Repositorio 5

   4.4 Paso 4: Estructuración y Redacción del Archivo README.md 5

   4.5 Paso 5: Registro de Cambios y Confirmación (Commit Changes) 6

   4.6 Paso 6: Inicialización Local y Sincronización Remota (Push) 6

5. Gestión de Licencias en GitHub 7

6. Fuentes de Información y Referencias 8

7. Conclusiones Individuales y de Equipo 9

8. Evidencias de Repositorios y Commits (Capturas GitHub - karoAH) . 10

9. Evidencia de Interfaz JavaFX 11

10. Cuestionario de la Práctica 12

1. INTRODUCCIÓN

En el desarrollo de software, el control de versiones y el trabajo colaborativo constituyen pilares fundamentales para garantizar la integridad, la trazabilidad y la calidad del código fuente. GitHub se ha consolidado como la plataforma en la nube más utilizada para alojar proyectos de desarrollo de software, permitiendo a los desarrolladores gestionar repositorios, colaborar de manera distribuida y mantener un registro histórico exacto de cada cambio realizado en el proyecto.

El presente reporte de práctica documenta detalladamente la ejecución del proceso de configuración, inicialización y gestión de un repositorio remoto utilizando la herramienta de línea de comandos Git en integración con GitHub. A lo largo de este informe se integran las evidencias fotográficas y capturas de pantalla obtenidas durante el desarrollo de la actividad en el laboratorio, respaldando el cumplimiento de cada instrucción técnica y mostrando la ejecución paso a paso del programa y los comandos implementados.

2. OBJETIVO DE LA PRÁCTICA

El objetivo principal de esta práctica es que las estudiantes desarrollen habilidades técnicas fundamentales en el control de versiones y gestión de proyectos de software mediante el cumplimiento de las siguientes metas específicas:

1. Crear y configurar correctamente una cuenta de usuario en la plataforma GitHub.
2. Descargar, instalar y configurar la herramienta Git en el sistema operativo local.
3. Crear y estructurar un nuevo repositorio remoto en GitHub aplicando convenciones de nombres válidas (caracteres ASCII).
4. Vincular el entorno local de desarrollo con el repositorio en la nube a través de Git Bash o PowerShell.
5. Subir un proyecto de software al repositorio e implementar mensajes de confirmación (commits) explicativos.
6. Actualizar y sincronizar cambios en el proyecto manteniendo el control de ramas principales (main branch).

3. MATERIALES Y REQUERIMIENTOS DEL SISTEMA

Para la ejecución satisfactoria de esta práctica se requirieron los siguientes materiales y recursos tecnológicos:

- **Computadora de Escritorio / Laptop:** Con sistema operativo Microsoft Windows y conexión a Internet.
- **Navegador Web:** Google Chrome, Mozilla Firefox o Microsoft Edge para el acceso al entorno web de GitHub.
- **Software Git Instalado:** Cliente de control de versiones descargado e instalado desde su sitio oficial (git-scm.com).
- **Terminal de Comandos:** Git Bash o Windows PowerShell para la interacción por línea de comandos.
- **Proyecto de Software / Archivos de Práctica:** Código fuente local y archivos de documentación (como README.md) para subir al repositorio.

4. DESARROLLO DE LA PRÁCTICA

A continuación se describe la secuencia sistemática de pasos realizados por el equipo de trabajo para completar la configuración e integración del proyecto en GitHub.

PASO 1: Creación de la Cuenta en GitHub

Se accedió al portal web oficial github.com para llevar a cabo el proceso de registro institucional. Se completó el formulario de registro y, posterior a la verificación de la dirección de correo, el sistema desplegó la confirmación de inicio de sesión.

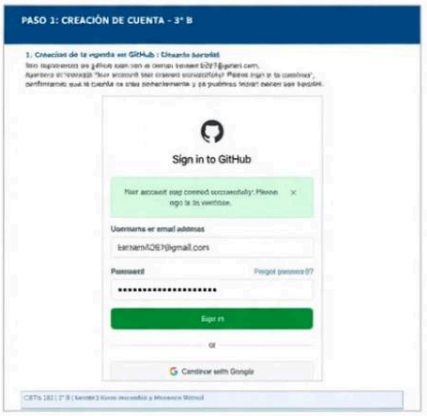


Figura 1: Confirmación de registro exitoso e inicio de sesión en GitHub.

CBTis 182 | 3° B | Karen Jacquelin y Alexandra Marisol

PASO 2: Instalación y Configuración Inicial de Git

Se realizó la descarga del instalador oficial de Git desde git-scm.com y se ejecutó en la computadora. Una vez en el panel principal de GitHub, la plataforma presentó las guías de inicio rápido como "Create your first code project", "GitHub Documentation" y "How to create your first repository", las cuales brindaron el marco de referencia práctico.

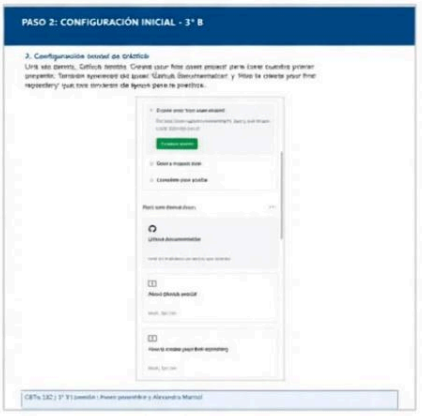


Figura 2: Panel de bienvenida de GitHub con accesos a la documentación inicial de la práctica.

CBTis 182 | 3° B | Karen Jacquelin y Alexandra Marisol

PASO 3: Creación y Ajuste de Nombre del Repositorio

Se procedió a crear el repositorio remoto. Al configurar el nombre como `Practica2-gestion-proyectos`, GitHub confirmó la disponibilidad del nombre (*Practica2-gestion-proyectos is available*) y permitió establecer la rama principal predeterminada (*Default branch*) como `main`.



Figura 3: Configuración del repositorio remoto con los parámetros de visibilidad pública.

CBTis 182 | 3° B | Karen Jacquelin y Alexandra Marisol

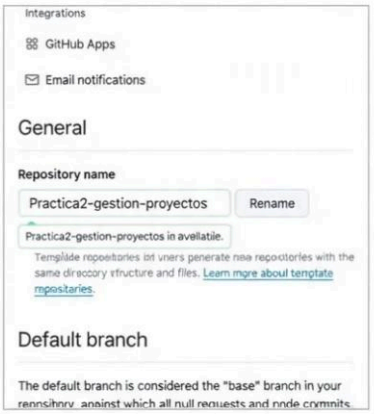


Figura 4: Verificación del nombre 'Practica2-gestion-proyectos' en la sección General de GitHub.

CBTis 182 | 3° B | Karen Jacquelin y Alexandra Marisol

PASO 6: Inicialización Local y Sincronización Remota (Push)

En la terminal local de Git Bash se establecieron los parámetros globales de la usuaria y s ejecutó la vinculación con el servidor remoto para realizar el envío definitivo del código (push).

```
git config --global user.name "Karen Jacquelin"
git config --global user.email "jacquelinegr202206@gmail.com"

# Inicialización y preparación de la carpeta local
git init
git add .
git commit -m "Initial commit - Estructura inicial de la Practica 2"
git branch -M main

# Vincular y sincronizar con el repositorio remoto de GitHub
git remote add origin https://github.com/Practica2-gestion--proyectos.git
git push -u origin main
```

5. GESTIÓN DE LICENCIAS EN GITHUB

¿Qué es una licencia en GitHub?
La licencia es un archivo en un repositorio que especifica los permisos y restricciones legales que tienen otras personas con respecto al proyecto, indicando si pueden copiarlo, modificarlo, distribuirlo o comercializarlo.

Resumen de Licencias de Software Principales:

1. MIT License: Es la más utilizada en la industria. Permite que cualquiera use, copie, modifique y venda el código, con la única condición de mantener la nota de derechos de autor y dar crédito.

2. Apache License 2.0: Similar a MIT, pero otorga protección explícita contra demandas por patentes y exige documentar las modificaciones realizadas al código original.

3. GPL v3 (General Public License): Licencia copyleft estricta. Obliga a que cualquier proyecto derivado o que use el código sea distribuido de manera gratuita y bajo código abierto.

4. BSD License (2 y 3 cláusulas): Muy permisiva y parecida a MIT. Permite la redistribución y modificación comercial siempre que se otorgue crédito al autor original.

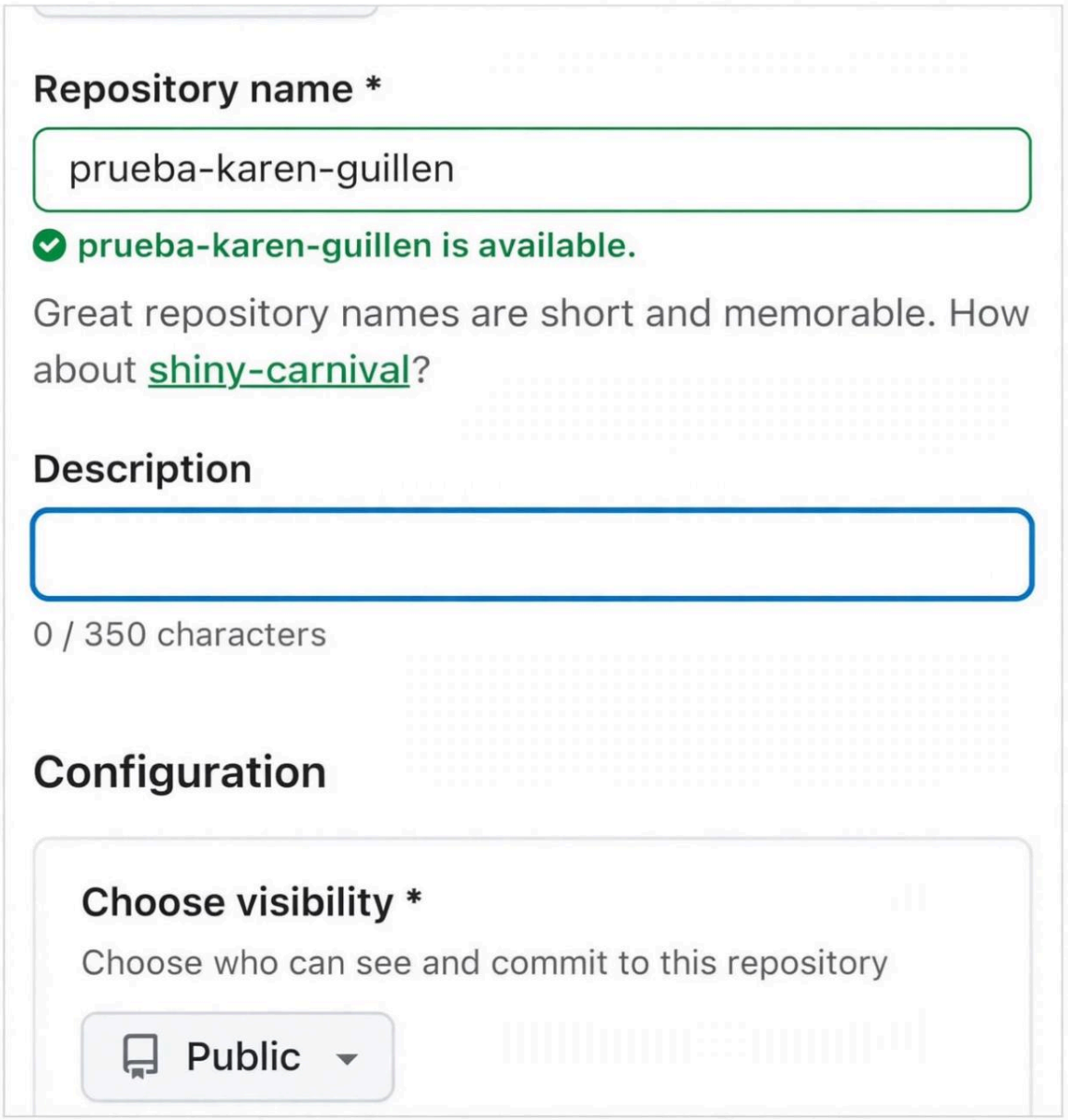
5. Creative Commons (CC BY 4.0): Orientada a documentación, imágenes y textos del reporte en lugar de código ejecutable, permitiendo su compartición otorgando atribución.

6. Sin Licencia (None): Al no asignar ninguna licencia, aplican por defecto las leyes de derechos de autor tradicionales (*All rights reserved*). Nadie tiene permiso legal para usar ó modificar el código sin autorización explícita.

¿Cuál utilizamos en nuestro proyecto?

Para la **Práctica** 2 se configuró la opción **None (Sin licencia)** según las indicaciones del Prof. Jaime Peregrino, asegurando que el contenido del repositorio permanezca como propiedad privada y de uso exclusivo del equipo escolar.

1. PASO 1: Creacion repo - Karen



Descripcion:
Se ingreso con cuenta karoAH y se verifico disponibilidad.
Sistema confirmo en verde: 'prueba-karen-guillen is available'.
Se configuro como Publico.

2. PASO 1: Creacion repo - Alexandra

Required fields are marked with an asterisk (*).

1 General

Owner *

karoAH

Repository name *

prueba-alexandra-jacobo

prueba-alexandra-jacobo is available.

Great repository names are short and memorable. How about ideal-octo-memory?

Description

Repositorio de prueba - Alexandra Marisol Jacobo I

61 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. About READMEs

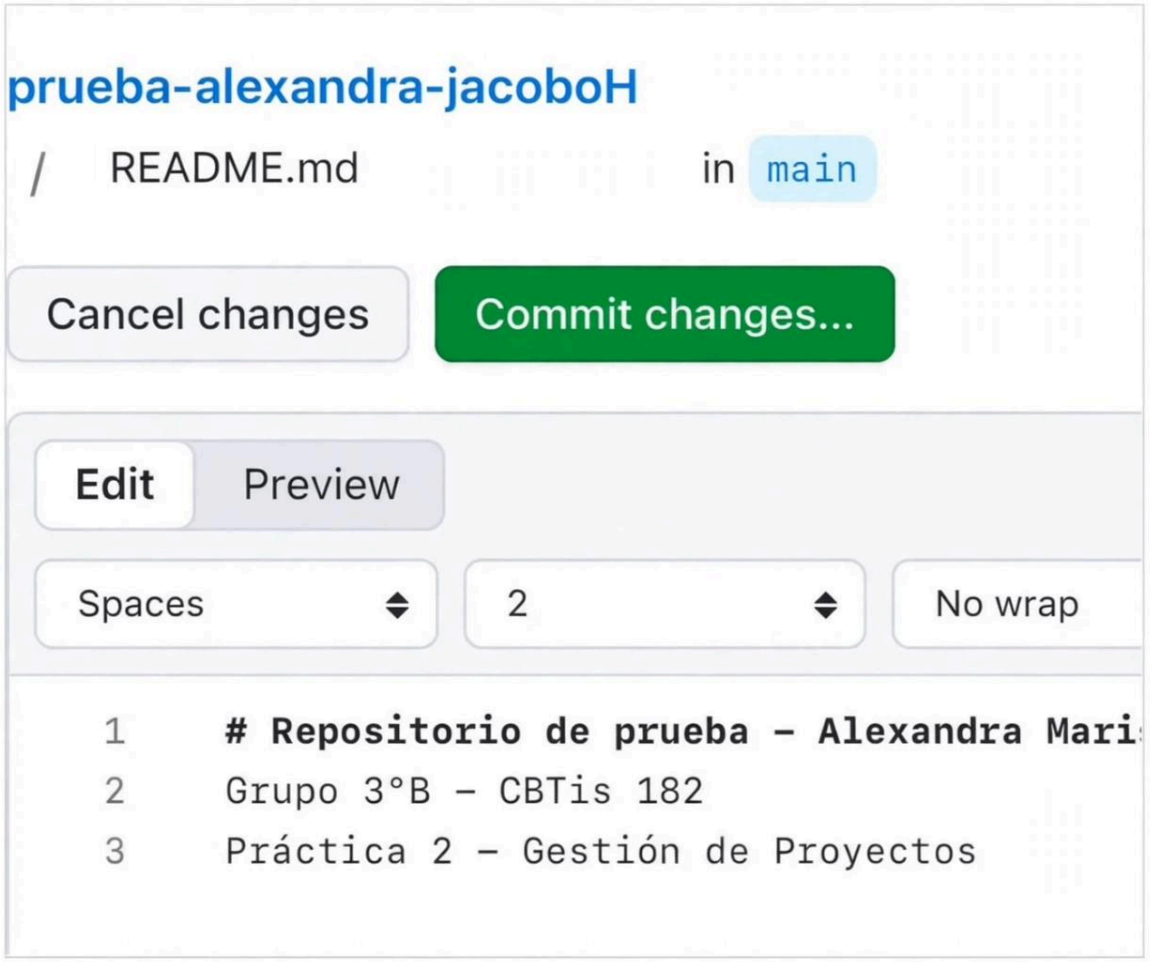
Off

Add .gitignore

.gitignore tells git which files not to track. About ignoring files

Descripcion:

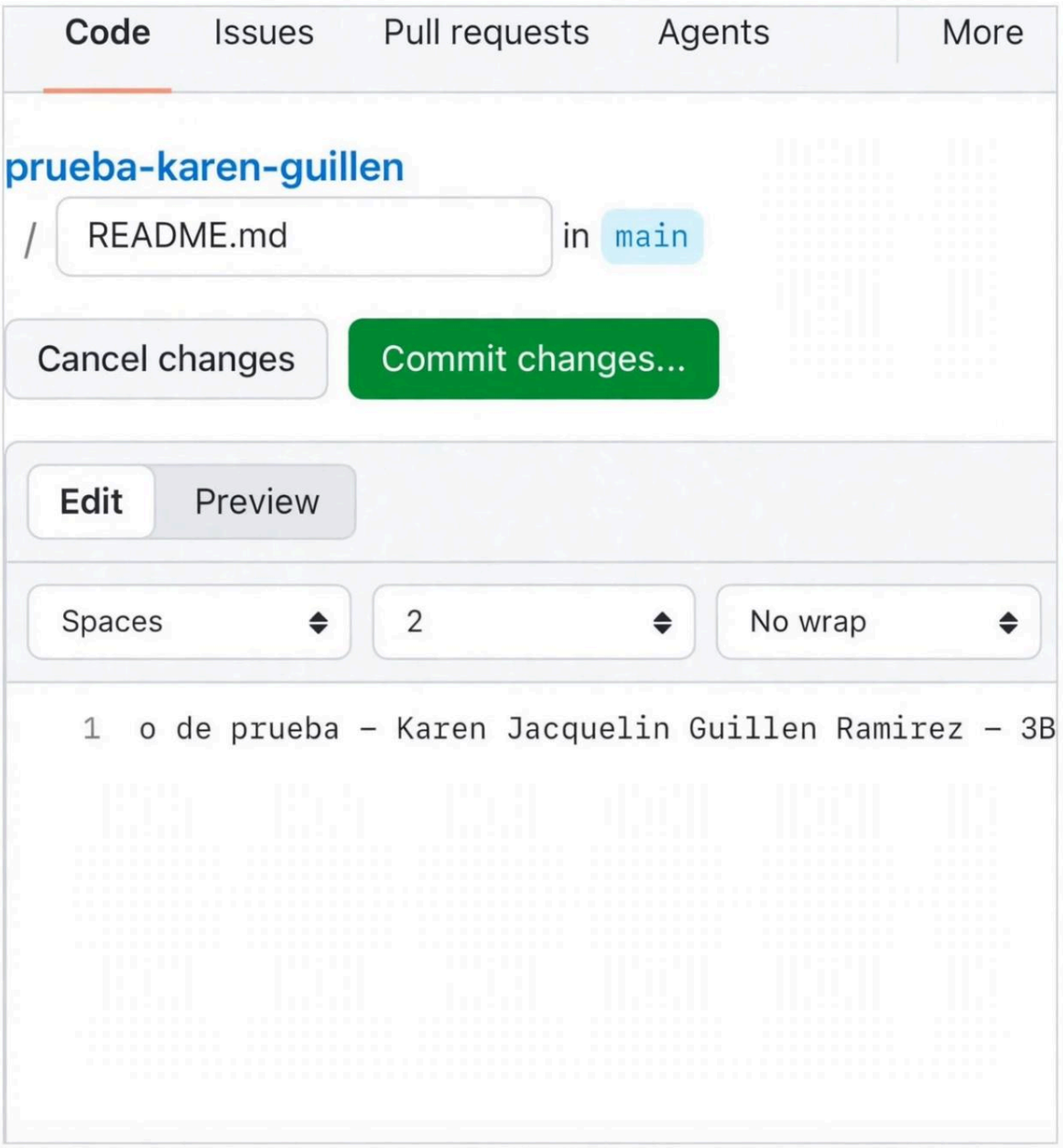
3. PASO 2: Quick Setup - Repo vacio



Descripcion:

Pantalla inicial de GitHub al crear repo vacio. Muestra opciones para crear nuevo archivo, subir archivo o usar comandos git init, add, commit y push.

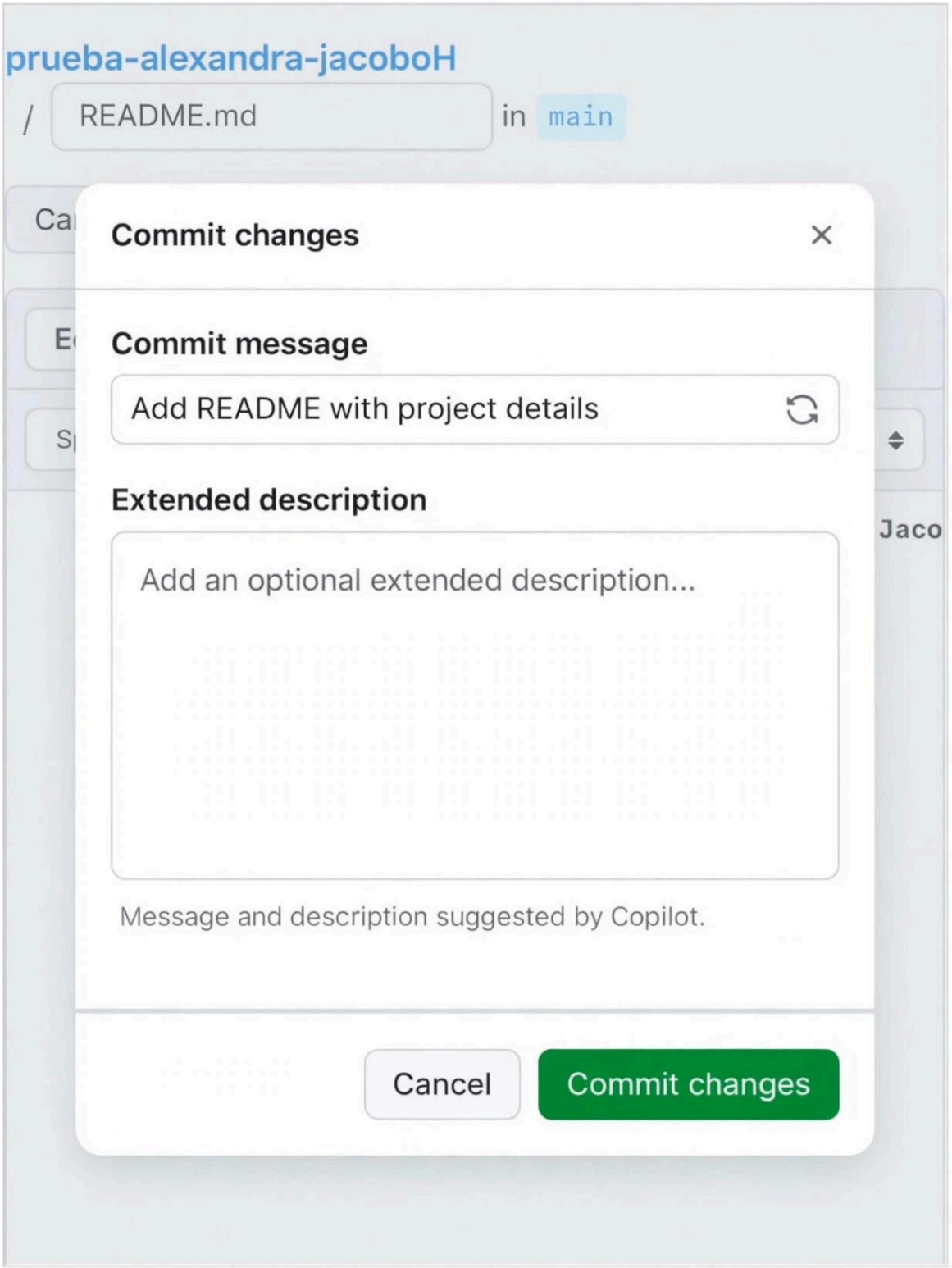
5. PASO 3: Edicion README - Alexandra



Descripcion:

Se nombro archivo como README.md en rama main. Se ve contenido con datos de Alexandra Jacobo y boton Commit changes.

6. PASO 3: Guardado (Commit)



Descripcion:
Ventana confirmacion. Se coloco mensaje 'Add README with project details' que queda en historial como primer commit.

7. PASO 3: Creacion README - Karen



Add collaborators to this repository

Search for people using their GitHub username or email address.

Invite collaborators

Quick setup — if you've done this kind of thing before

HTTPS | SSH | https://github.com/ka

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

...or create a new repository on the command line

```
echo "# prueba-alexandra-jacoboH"
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://git
git push -u origin main
```

...or push an existing repository from

8. PASO 4: Resultado Final

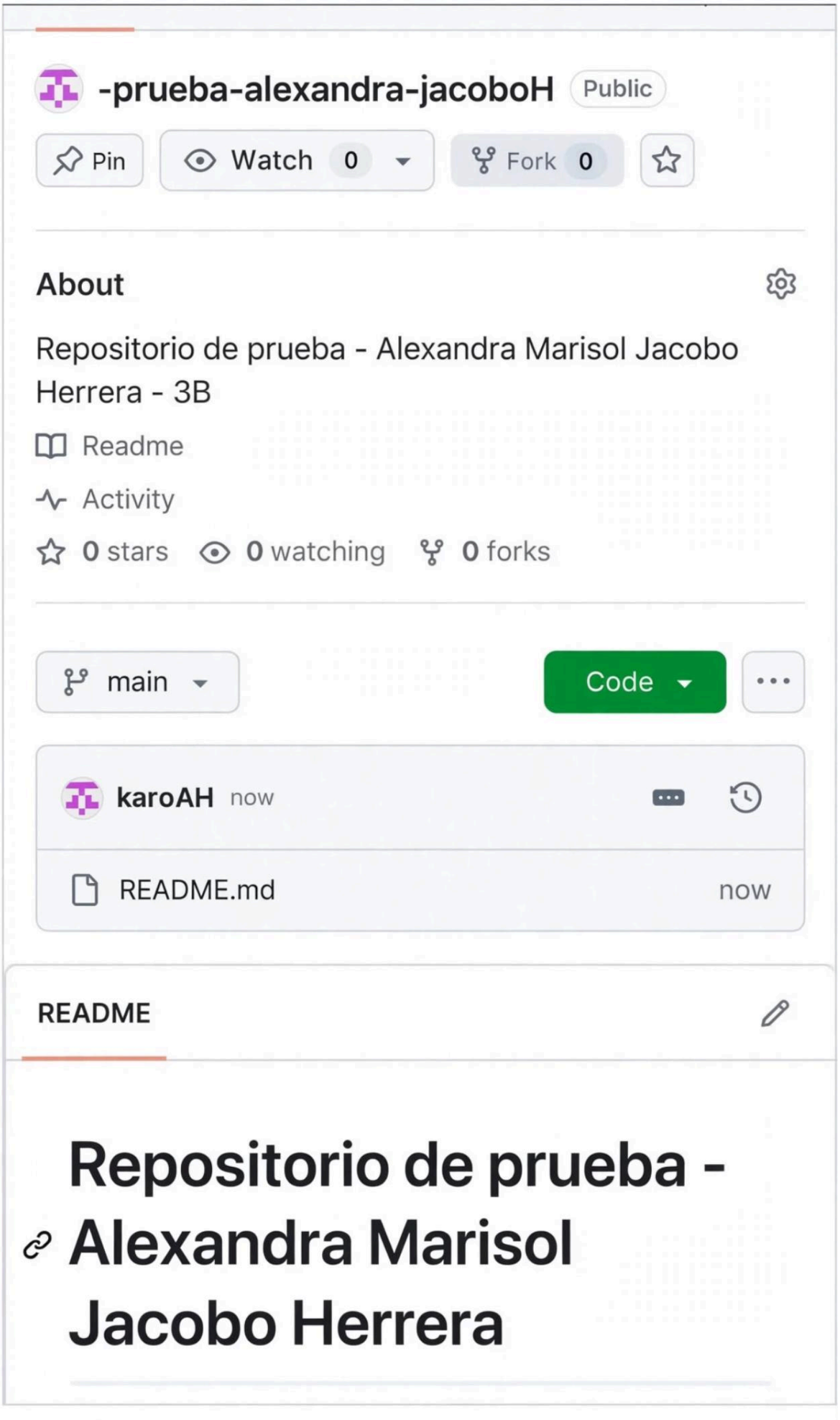


Figura 1 - Ejecución de HelloApplication.java

En esta evidencia se muestra la ejecución correcta del proyecto ProyectoJavaFX en intellij SDK. Se crea la clase principal HelloApplication.java que extiende de Application y se configura el metodo start() para cargar hello-view.ftvn. Al ejecutarse se despliega la ventana con el titulo "Hola!" y un boton centrado, confirmando la integracion de JavaFX con el SDK. la parte superior se agrego el comentario // Author: Karen Guillen - Proyet: ProyectoJavaFX.

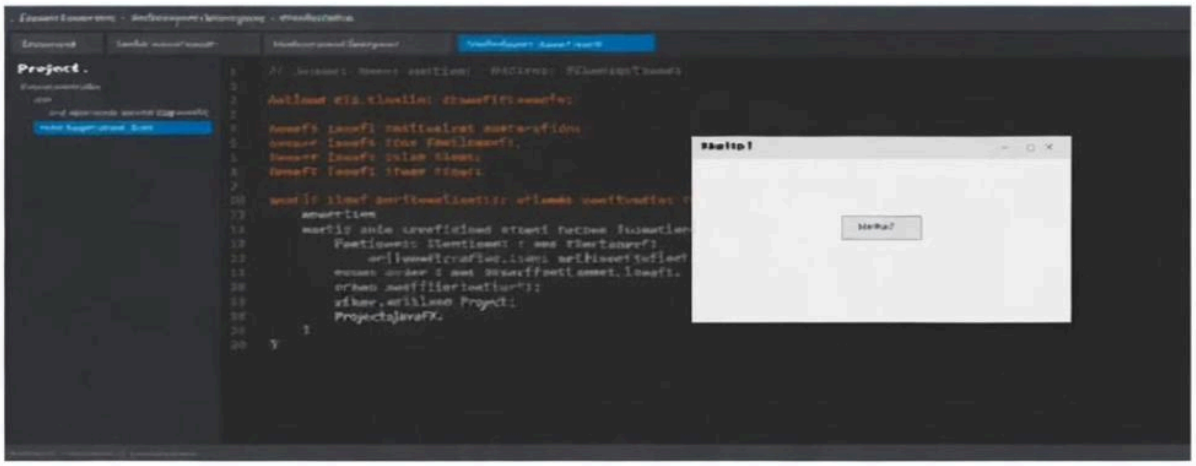


Figura 2 - Configuracion Run/Debug con OpenJDK-26

Para resolver el error inicial, se ingreso a Run/Debug Configurations. En Build and Run se establece OpenJDK26, en mismo configuraciones personales previas. En VM options se agreg --enable-native-access=Javafx.graphics, necesario para permitir el acceso nativo derivad de los modulos JavaFX. Con esto se corrige el problema de Restricted methods blocked.

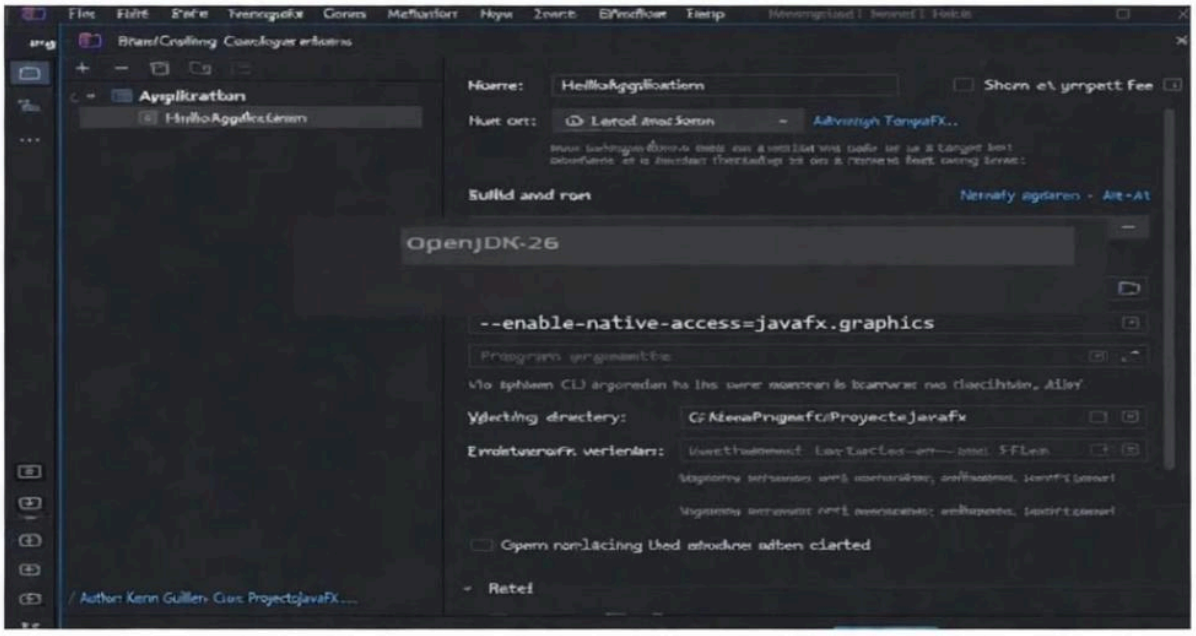


Figura 3 - Codigo fuente y consola de ejecucion

Para resolver el error inicial, vetudar del proyecto con la estructura org Build and kun se estableció y OpenJDK26, el m mismo configuraciones personales previas el En VM options se agrego --enable-native-access=Javafx.graphics, necesario para permitir el acceso nativo de los modulos JavaFX Con esto se corrige el problema de Restricted methods blocked.

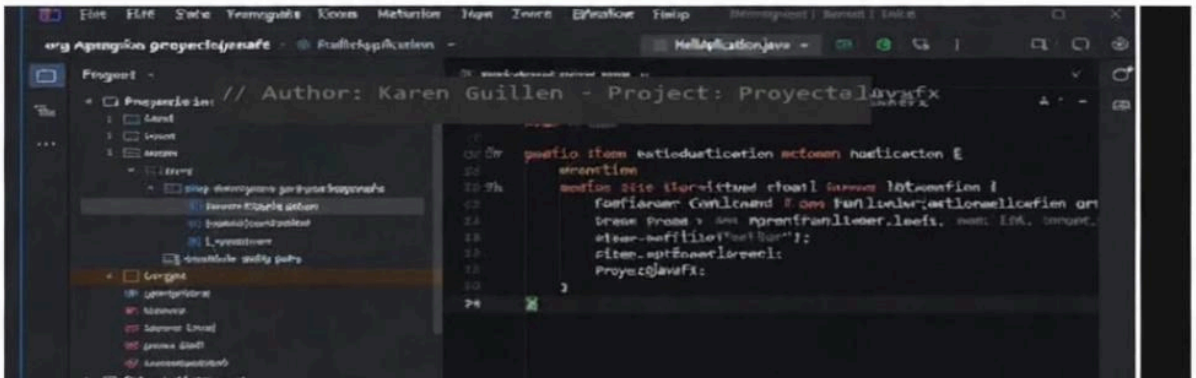




Figura 2 - 2. EDICIÓN DE CÓDIGO

Se editó Main.java, se modificó el mensaje de salida y se vinculó el proyecto a GitHub con VCS > Share Project on GitHub.
La notificación "Successfully shared project" confirma la vinculación.

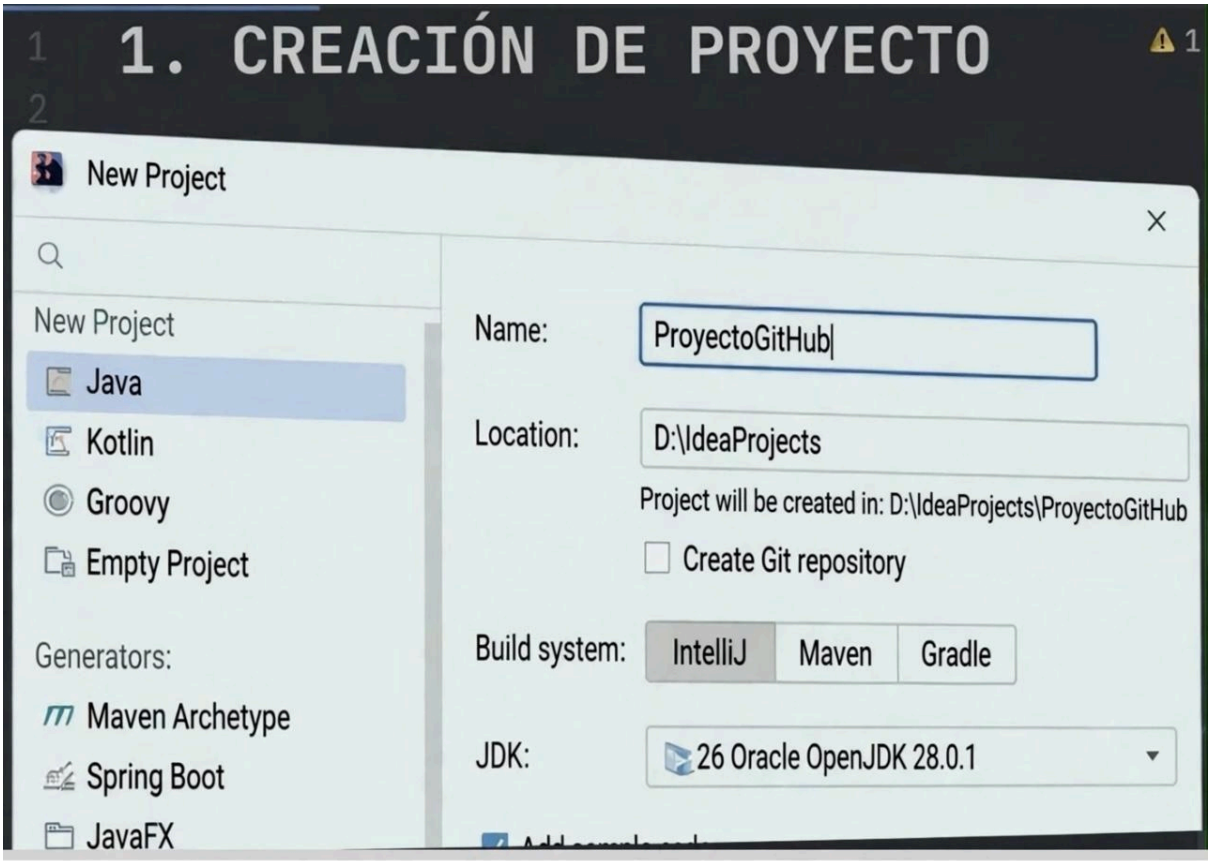


Figura 1 — 1. CREACIÓN DE PROYECTO

1. **File** > New **Project**
2. Seleccionar Java
3. Name: ProyectoGitHub
4. Location: **D:**\IdeaProjects
5. Build system: IntelliJ
6. JDK: 26 Oracle OpenJDK 28.0.1
7. Activar Add sample code
8. Click **Create**

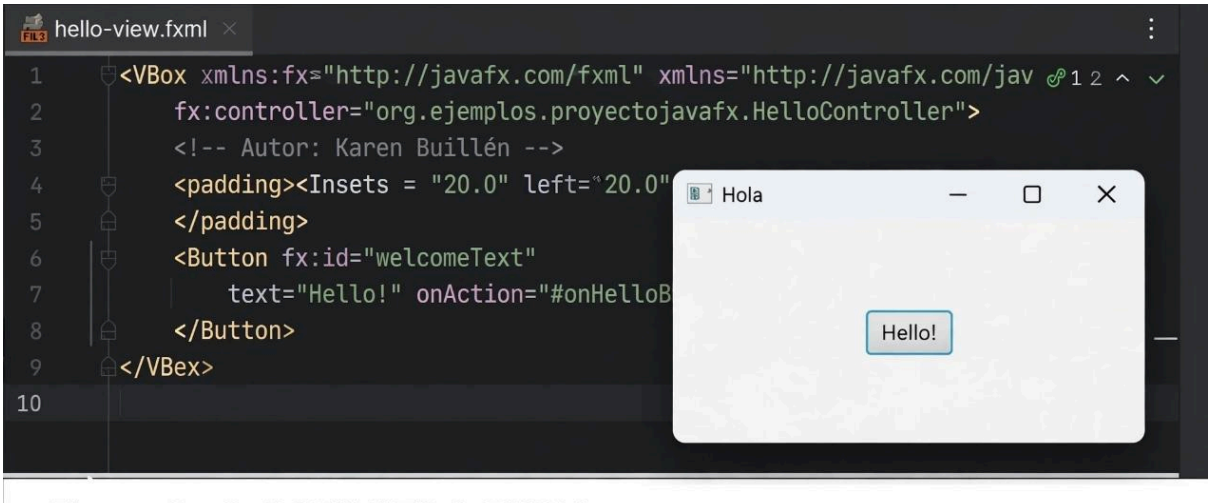


Figura 3 - 3. COMMITS & PUSH

Descripción: En la ventana Commit se escriben los mensajes: Primer Commit y Segundo Commit: Cambié los textos. Luego se presiona Commit and Push para enviar los cambios a GitHub.

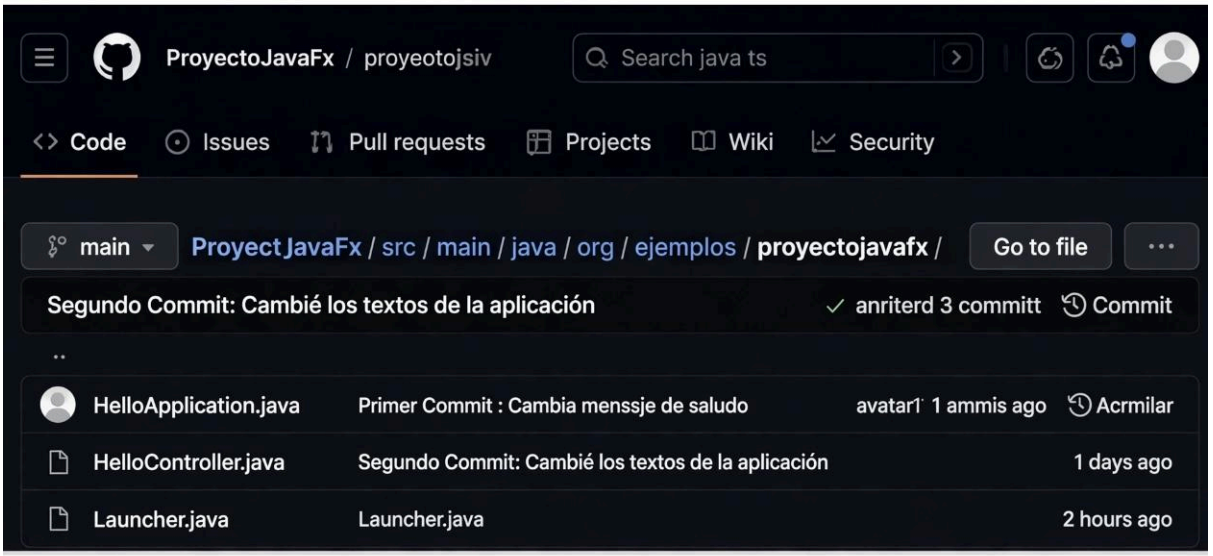


Figura 4 - EJECUCIÓN DE INTERFAZ

Se ejecutó HelloApplication.java con Run. Se muestra el archivo hello-view.fvwv y la ventana emergente "Hola" con botón Hello! que confirma que JavaFX funciona correctamente.

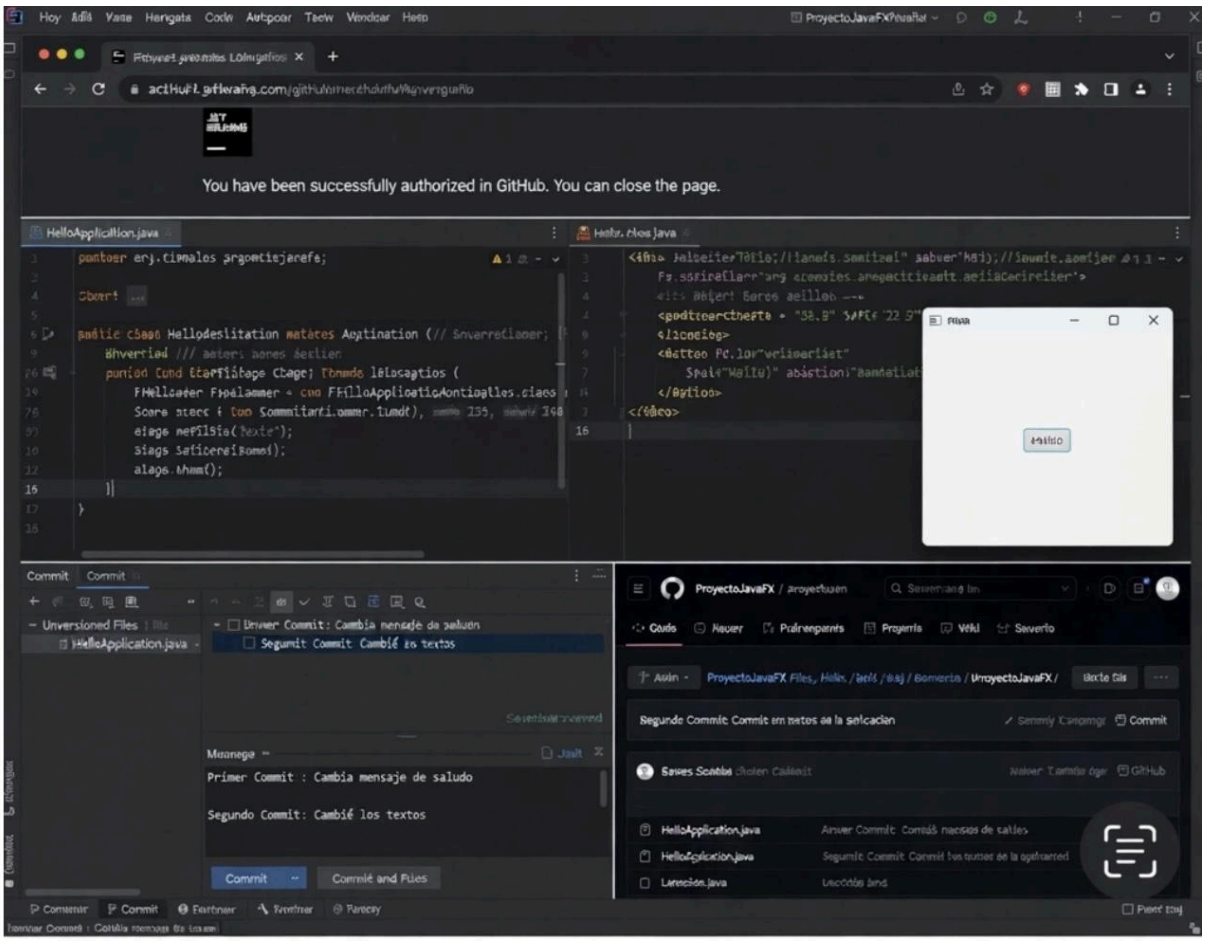


Figura 6 - HISTORIAL DE COMMITS LOCAL

En IntelliJ, pestaña Commit, se visualizan Unversioned Files, HelloApplication.java y los mensajes: Primer Commit: Cambia mensaje de saludo y Segundo Commit: Cambié los textos antes de hacer Commit and Push.

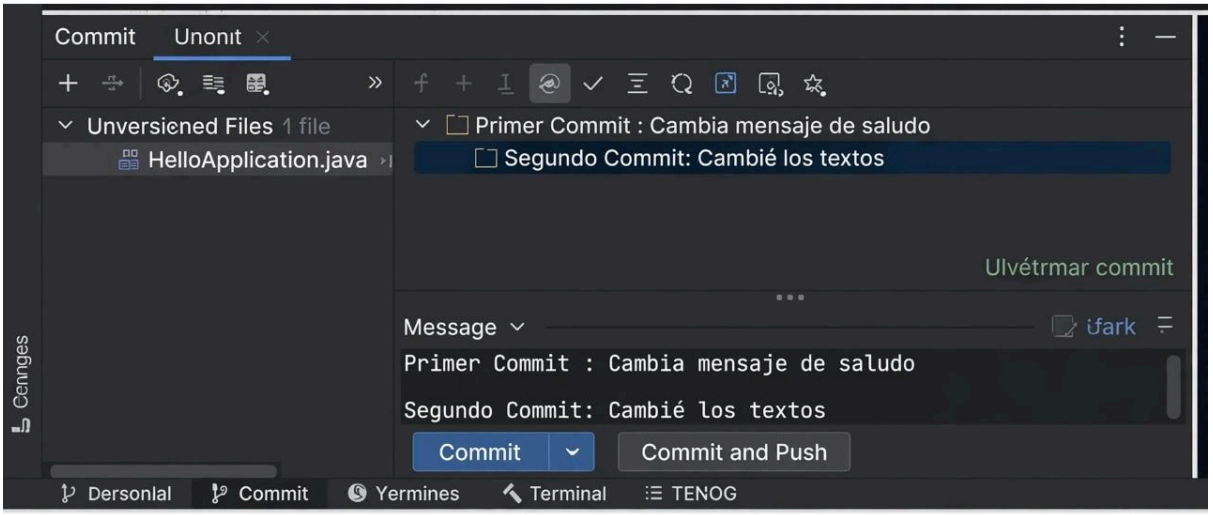


Figura 5 - VERIFICACIÓN EN GITHUB

En github.com se verifica el repositorio **ProyectoJavaFx.** Se observan los archivos HelloApplication.java, **HelloController.java** y **Launcher.java** con los commits: Primer Commit y Segundo Commit: **Cambié los** textos de la aplicación por **K**aren Guillén.

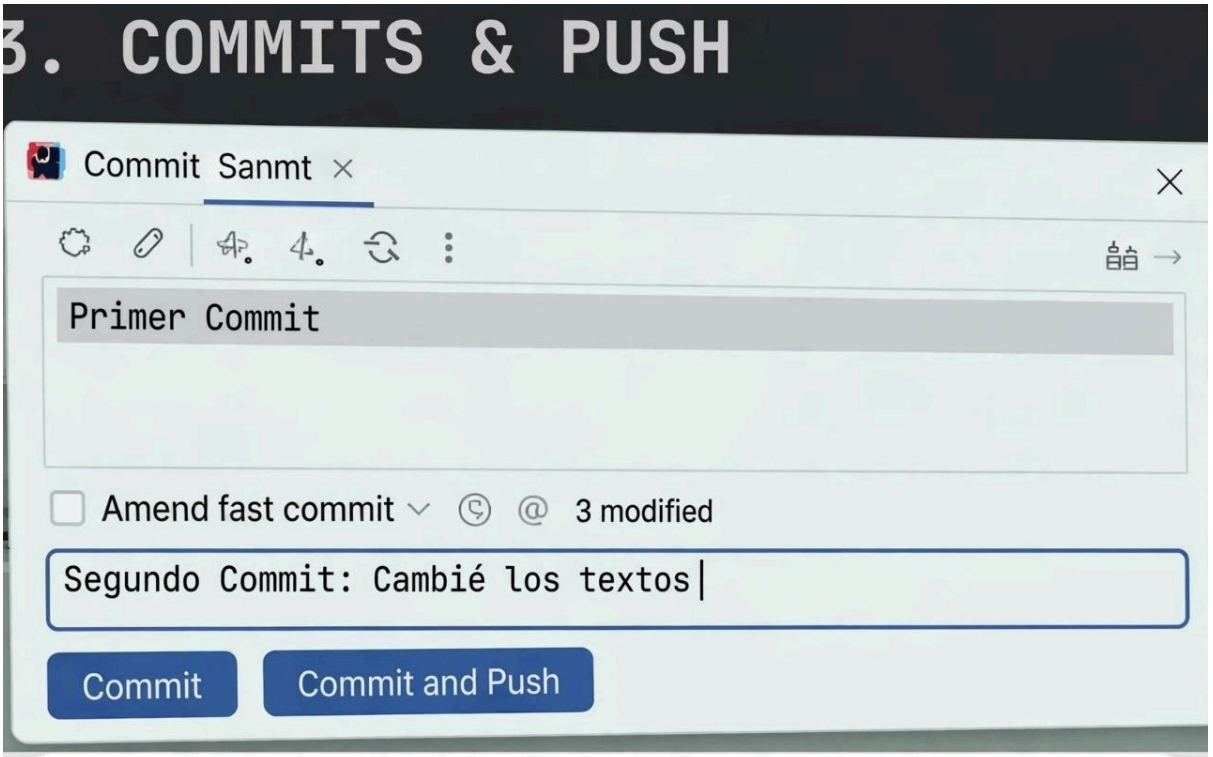


Figura 7 - AUTORIZACIÓN GITHUB + EVIDENCIA COMPLETA

Descripción: Arriba: Autorización exitosa en GitHub con JetBrains. Centro: Código HelloApplication.java y hello-view.ftiv con app Hola ejecutándose. Abajo: Panel Commit y vista en GitHub. Flujo completo verificado.

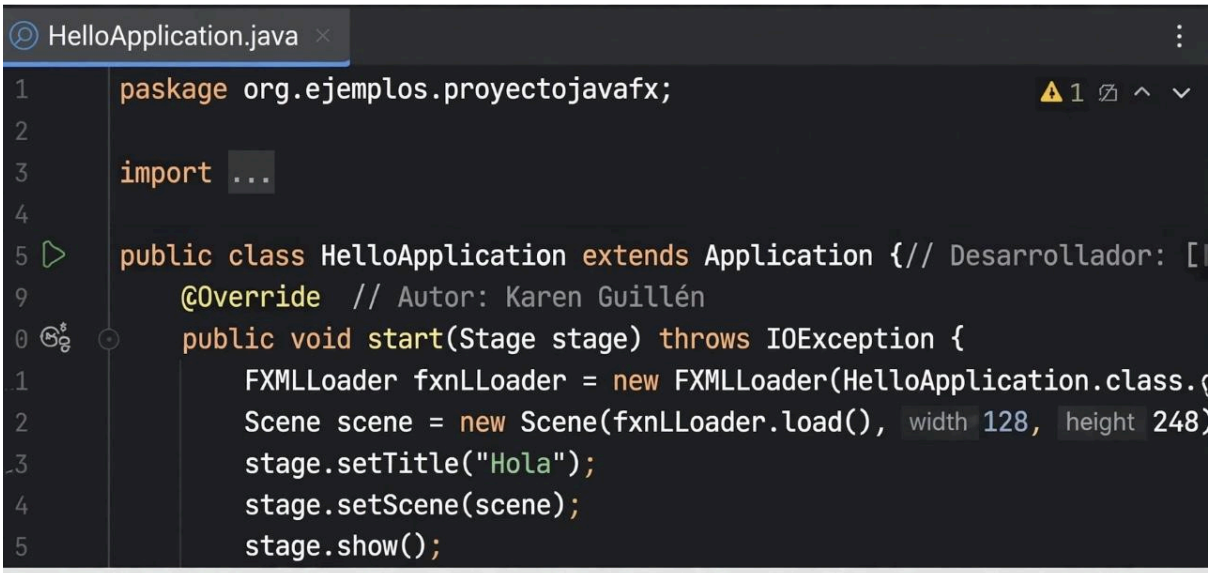


Figura 8 - DOCUMENTACIÓN DE CÓDIGO

Descripción: En HelloApplication.java se agregó el comentario // Autor: Karen Guillén. En el método start() se carga Floxdeader, Scene y Stage para mostrar la ventana principal. Se guarda el archivo.

7. FUENTES DE INFORMACIÓN Y REFERENCIAS

Para la realización exitosa de las actividades y la fundamentación técnica de este reporte, se consultaron las siguientes referencias bibliográficas e institucionales:

- **Git SCM Documentation:** Guía de referencia y manual oficial de comandos de Git. Disponible en: https://git-scm.com/doc
- **GitHub Docs:** Documentación oficial sobre la creación de repositorios y gestión de proyectos colaborativos. Disponible en: https://docs github.com/es
- **CBTis No. 182:** Apuntes de clase y guía práctica del Módulo M2S1 "Emplea Frameworks para el Desarrollo de Software", impartido por el Prof. Jaime Peregrino, Zinapécuaro, Michoacán.

8. CONCLUSIONES INDIVIDUALES Y DE EQUIPO

Conclusión de Alexandra Marisol Jacobo Herrera:

"En el desarrollo de esta práctica logré comprender la importancia real que tiene el control de versiones en nuestra formación como desarrolladoras de software. La plataforma GitHub y la herramienta Git no solo nos ayudan a respaldar nuestro código en la nube de forma segura, sino que facilitan el trabajo en equipo al sincronizar avances sin perder información. Esta experiencia me dio mayor confianza para gestionar proyectos escolares futuros dentro de la especialidad."

Conclusión de Karen Jacquelin Guillén Ramírez:

"A través de los pasos realizados en la práctica pude dominar el flujo de trabajo esencial con los comandos básicos de la terminal de Git, desde la inicialización de un repositorio local hasta el envio de actualizaciones mediante push. Considero que aprender a utilizar estas herramientas profesionales en el módulo M2S1 es clave para preparamos adecuadamente hacia el desarrollo de aplicaciones con frameworks y el trabajo colaborativo en la industria."

Conclusión General del Equipo:

Como equipo de trabajo del 3° Semestre Grupo B, concluimos que la práctica cumplió satisfactoriamente con todos los objetivos planteados. Logramos integrar nuestras competencias individuales en el uso de terminales y entornos web, consolidando un flujo de trabajo ágil, limpio y profesional que servirá como base para los subsecuentes módulos de la especialidad en desarrollo de software.

Alumnos(as): Karen Jacquelin Guillén Ramírez & Alexandra Marisol Jacobo Herrera
Grado y Grupo: 3° B • Programación

Práctica 2 - Gestión de Proyectos: Control de Versiones con Git y GitHub

1. ¿Qué es y para qué sirve GitHub? ¿Qué es y para qué sirve git? ¿Cuáles son las diferencias entre ellos? ¿Cómo se complementan entre sí?

- **¿Qué es y para qué sirve Git?** Es un sistema de control de versiones distribuido diseñado para rastrear cambios en el código fuente durante el desarrollo de software. Sirve para registrar el historial de modificaciones, permitir el trabajo en paralelo mediante ramas y facilitar la reversión a estados anteriores en caso de errores.
- **¿Qué es y para qué sirve GitHub?** Es una plataforma web basada en la nube que aloja repositorios Git. Sirve para centralizar el código en servidores remotos, facilitando la colaboración en equipo, la gestión de tareas (Issues, tableros Kanban) y la revisión de código (Pull Requests).
- **Diferencias principales:** *Git* es la herramienta de software local que opera en la computadora para gestionar versiones de forma local, mientras que *GitHub* es un servicio web alojado en internet que sirve como punto de encuentro y almacenamiento remoto para dichos repositorios.
- **Cómo se complementan:** Git maneja toda la lógica de control de versiones de manera local en el equipo (commits, ramas, fusiones), y GitHub actúa como el repositorio centralizado en la nube donde se respaldan, comparten y co-gestionan esos cambios con otros desarrolladores.

2. ¿Cuáles son las ventajas de usar GitHub para gestionar proyectos?

- **Colaboración en equipo:** Permite que múltiples desarrolladores trabajen de forma simultánea en un mismo proyecto sin sobrescribir el trabajo de los demás mediante sistemas de ramas y fusiones ordenadas.
- **Revisión de Código (Code Review):** Facilita la discusión y validación de cambios mediante Pull Requests antes de integrarlos a la versión principal.
- **Control de acceso y seguridad:** Ofrece la posibilidad de configurar repositorios públicos o privados y gestionar permisos detallados para los colaboradores.
- **Gestión ágil de proyectos:** Incorpora herramientas integradas como tableros Kanban, asignación de tareas (Issues), seguimiento de errores y documentación automatizada (GitHub Pages / README).
- **Respaldo en la nube:** Asegura que el código fuente está resguardado de manera segura en servidores externos ante cualquier fallo del equipo local.

3. ¿Qué proceso se siguió para configurar y/o instalar GitHub y/o Git? Incluir detalles y capturas de pantalla que comprueben el proceso seguido, explicar brevemente cada captura.

Para la instalación y configuración de Git en el equipo local y su vinculación con GitHub, se siguió el siguiente procedimiento técnico:

- **Descarga e Instalación de Git:** Se descargó el instalador oficial desde el sitio web de Git (`git-scm.com`). Durante el asistente de instalación, se seleccionaron las opciones recomendadas (como usar Git Bash por defecto, habilitar Git desde la terminal de comandos y configurar los saltos de línea automáticos).
- **Configuración Inicial de Identidad:** Una vez instalado Git Bash, se configuraron las credenciales de usuario mediante comandos globales en la terminal para firmar los commits:
  - `git config --global user.name "Karen Jacquelin Guillén Ramirez"`
  - `git config --global user.email "correo@ejemplo.com"`
- **Vinculación con GitHub y Autenticación:** Se creó una cuenta activa en la plataforma GitHub (`github.com`). Para permitir la comunicación segura entre el equipo local y el repositorio remoto, se generó un token de acceso personal (Personal Access Token) o se configuraron las llaves SSH, vinculándolas en la sección de configuración de seguridad de la cuenta.

4. ¿Qué pasos se deben seguir para crear un repositorio?

La creación de un repositorio se puede realizar tanto en la plataforma web de GitHub como de manera local en el entorno de desarrollo:

- **Creación en GitHub (Repositorio Remoto):**
  1. Iniciar sesión en GitHub y hacer clic en el botón `+` ubicado en la esquina superior derecha, seleccionando *"New repository"*.
  2. Asignar un nombre descriptivo al repositorio (por ejemplo, `kaluna-recycle`).
  3. Seleccionar la visibilidad del repositorio (Público o Privado).
  4. Marcar la opción de inicializar el repositorio con un archivo `README.md` y, opcionalmente, agregar un archivo `.gitignore` (según el lenguaje de programación, como Java o NetBeans) y una licencia.
  5. Hacer clic en *"Create repository"*.
- **Creación Local mediante Git:**
  1. Abrir la terminal o Git Bash en la carpeta del proyecto.
  2. Ejecutar el comando `git init` para inicializar el repositorio local.
  3. Vincular el repositorio local con el remoto ejecutando `git remote add origin <URL-del-repositorio>`.

5. ¿Qué es una rama y para qué sirve crear una rama?

Una **rama (branch)** en Git es un apuntador móvil independiente hacia uno de los commits realizados en el proyecto. Representa una línea de desarrollo paralela separada de la rama principal (tradicionalmente llamada `main` o `master`).

¿Para qué sirve?

- Permite desarrollar nuevas funcionalidades, corregir errores o experimentar con el código fuente sin alterar ni poner en riesgo la estabilidad de la versión principal o de producción del software.
- Facilita el trabajo en equipo, ya que cada desarrollador o miembro del equipo puede trabajar en su propia rama de manera independiente y simultánea sin interferir con el código de sus compañeros.
- Permite realizar pruebas exhaustivas y revisiones de código antes de integrar los cambios de manera definitiva.

6. ¿Qué es un pull request, de git/github, para qué sirve y como se usa?

Un **Pull Request (PR)** es una propuesta formal de combinación de código dentro de GitHub. Ocurre cuando un desarrollador ha realizado cambios en una rama independiente y solicita que el administrador o los colaboradores del repositorio principal revisen, discutan e integren esos cambios en la rama principal (`main`).

¿Para qué sirve y cómo se usa?

- **Sirve para:** Fomentar la revisión de código colaborativa (Code Review), detectar errores o conflictos antes de fusionar el código, y mantener un registro documentado de las modificaciones introducidas en el proyecto.
- **Cómo se usa:**
  1. Se sube la rama de trabajo al repositorio remoto en GitHub (`git push origin nombre-rama`).
  2. En la intertaz web de GitHub, aparece una alerta para abrir un nuevo Pull Request. Se hace clic en *"Compare & pull request"*.
  3. Se redacta un título y una descripción clara detallando los cambios realizados.
  4. El equipo revisa los cambios, realiza comentarios si es necesario, y finalmente se aprueba y ejecuta la fusión (*Merge pull request*).

7. ¿Cómo se puede colaborar con otros desarrolladores a través de GitHub?

GitHub proporciona un ecosistema robusto para la colaboración en equipo mediante diversas estrategias y herramientas:

- **Añadir Colaboradores:** Desde la configuración del repositorio (`Settings > Collaborators`), se pueden invitar a otros compañeros de equipo para que tengan permisos de lectura y escritura directos.
- **Sistema de Ramas y Pull Requests:** Permite que cada integrante trabaje en características específicas y proponga fusiones ordenadas mediante revisiones conjuntas.
- **Gestión de Tareas (Issues y Projects):** Se pueden crear tableros Kanban (Proyectos), asignar tareas (Issues) a miembros específicos del equipo, etiquetarlas por prioridad (utilizando metodologías ágiles como Scrum y MoSCoW) y dar seguimiento al progreso del desarrollo.
- **Resolución de Conflictos:** Cuando dos desarrolladores modifican la misma línea de código, GitHub y Git alertan sobre los conflictos, permitiendo analizarlos y resolverlos de forma manual antes de integrar el software.

8. ¿Cuáles sạ las diferencias entre un repositorio público y uno privado?

La principal diferencia radica en el nivel de acceso, visibilidad y privacidad del código fuente almacenado:

- **Repositorio Público:**
  - Es visible para cualquier usuado de Internet en todo el mundo.
  - Cualquier persona puede clonar, descargar o inspeccionar el código (ideal para proyectos de código abierto u *open source* y portafolios académicos).
  - Permite que desarrolladores externos contribuyan mediante Pull Requests públicos.
- **Repositorio Privado:**
  - El acceso está estrictamente restringido al propietario del repositorio y a los colaboradores autorizados explícitamente.
  - El código fuente no puede ser visto, clonado ni accedido por personas ajenas al equipo de trabajo.
  - Es ampliamente utilizado en el ámbito profesional, empresarial y en proyectos escolares confidenciales para proteger la propiedad intelectual.

9. ¿Cómo se elimina un repositorio de github?

Para eliminar permanentemente un repositorio de la plataforma GitHub, se debe seguir el procedimiento administrativo de seguridad:

1. Ingresar a la plataforma web de GitHub y seleccionar el repositorio que se desea eliminar.
2. Hacer clic en la pestaña de **Settings** (Configuración) ubicada en la parte superior derecha del menú del repositorio.
3. Desplazarse hasta el final de la página general de configuración hasta encontrar la sección de **Danger Zone** (Zona de Peligro).
4. Hacer clic en el botón **"Delete this repository"** (Eliminar este repositorio).
5. Aparecerá una ventana de confirmación pidiendo verificar la acción. Se debe escribir exactamente el nombre completo del repositorio (por ejemplo, `usuario/nombre-repositorio`) en el cuadro de texto indicado para confirmar.
6. Hacer clic en el botón final de confirmación para completar la eliminación definitiva.

10. ¿Cómo se puede integrar el uso de Git/Github al entorno de desarrollo (IDE) usado en clases?

En las prácticas de programación se utilizan entornos de desarrollo como **IntelliJ IDEA** y **NetBeans**, los cuales cuentan con soporte nativo e integración directa para Git y GitHub:

- **Integración en IntelliJ IDEA:**
  - Configuración inicial: Ir a *File > Settings > Version Control > Git* y verificar la ruta del ejecutable de Git.
  - Conexión con GitHub: Ir a *File > Settings > Version Control > GitHub* e iniciar sesión mediante cuenta o Token de acceso.
  - Uso diario: Desde el menú superior *Git*, se pueden realizar directamente operaciones de *Commit, Push, Pull, Update* y gestión de ramas sin necesidad de salir del IDE o usar la terminal externa.
- **Integración en Apache NetBeans:**
  - Configuración y Clonación: Ir al menú *Team > Git* o utilizar la opción de clonar repositorio directamente al crear o abrir un proyecto.
  - Control de Versiones: NetBeans marca en colores los archivos modificados en la ventana de proyectos y permite realizar commits, diffs (comparaciones) y pull/push haciendo clic derecho sobre el proyecto y seleccionando el submenú *Git*.

Enlaces a Repositorios de GitHub de la Práctica

A continuación se presentan los enlaces directos a los repositorios creados y utilizados durante el desarrollo de la práctica:

- **Repositorio Principal de la Práctica:** Practica2-gestion-proyectos
- **Repositorio de Prueba (Alexandra Jacobo H.):** -prueba-alexandra-jacoboH
- **Repositorio de Prueba (Karen Guillén):** prueba-karen-guillen

6. CUESTIONARIO DE LA PRÁCTICA

1. ¿Qué es Git y cuál es la diferencia principal con GitHub?

Respuesta: Git es un sistema de control de versiones distribuido que se ejecuta localmente en la computadora para rastrear y gestionar los cambios en el código fuente. Por otro lado, GitHub es un servicio de alojamiento en la nube que permite almacenar repositorios de Git, facilitando el trabajo colaborativo, la gestión de proyectos y el respaldo remoto en la web.

2. ¿Para qué sirven los comandos git add y git commit?

Respuesta: El comando `git add` agrega los archivos modificados al área de preparación (*Staging Area*), indicando qué cambios serán incluidos en el siguiente guardado. El comando `git commit` confirma y guarda de forma definitiva esos cambios en el historial del repositorio local junto con un mensaje explicativo.

3. ¿Cuál es la función del comando git push?

Respuesta: Permite transferir y subir los commits registrados en el repositorio local hacia el repositorio remoto alojado en GitHub, manteniendo sincronizado el proyecto para todos los colaboradores del equipo.

4. ¿Por qué es importante el uso de control de versiones en el desarrollo de software empleando frameworks?

Respuesta: Porque en proyectos complejos que utilizan frameworks se manejan múltiples archivos y dependencias. El control de versiones permite revertir errores, trabajar en nuevas funcionalidades en ramas independientes sin alterar el código estable y colaborar de manera limpia y ordenada entre múltiples desarrolladores.